Oliver Krauss

OOP Fundamentals

ALD | MTD.ba VZ SS26

Hagenberg 06.03.2026

HAGENBERG | LINZ | STEYR | WELS

FH OBERÖSTERREICH

UNIVERSITY OF APPLIED SCIENCES UPPER AUSTRIA

What You Already Know

In Modern Code (Semester 1) you learned **procedural Java**:

- Variables: `int health = 100; double radius = 5.0;`
- Methods: `public static int computeDamage(int base, double crit)`
- Arrays: `int[] scores = new int[10];`
- Everything lived inside **one class** with **static methods**
- Data was passed around as **parameters**

This semester: We move from "static methods with parameters" to **objects that know their own data**.

The Procedural Approach

How you would compute the area of a circle procedurally:

```
1   public class CircleCalculator {
2       static double calcArea(double radius) {
3           return Math.PI * radius * radius;
4       }
5       static double calcPerimeter(double radius) {
6           return 2 * Math.PI * radius;
7       }
8       public static void main(String[] args) {
9           double radius = 5.0;
10          double area = calcArea(radius);
11          IO.println("Area: " + area);
12      }
13  }
```

What do you notice about how data flows through this program?

A Procedural Game

A simple game with players and enemies -- the procedural way:

```
1   public class Game {
2       static int attack(int enemyHp, int damage) {
3           return Math.max(0, enemyHp - damage);
4       }
5       static boolean isDead(int hp) {
6           return hp <= 0;
7       }
8       public static void main(String[] args) {
9           String playerName = "Hero";
10          int playerHp = 100;
11          int playerDamage = 25;
12          String enemyName = "Goblin";
13          int enemyHp = 50;
14          int enemyDamage = 10;
15          enemyHp = attack(enemyHp, playerDamage);
16      }
17  }
```

The Procedural Problem

What happens when the game grows?

```
// Player data: 6 separate variables
String playerName = "Hero";
int playerHp = 100; int playerMaxHp = 100;
int playerDamage = 25; int playerArmor = 10;
int playerXp = 0;

// Enemy data: 6 more separate variables
String enemyName = "Goblin";
int enemyHp = 50; int enemyMaxHp = 50;
int enemyDamage = 10; int enemyArmor = 5;
int enemyXp = 20;

// Functions need ALL the data passed in
static int attack(int targetHp, int targetArmor, int damage) {...}
static String getStatus(String name, int hp, int maxHp) {...}
```

12 variables, growing parameter lists, no clear grouping -- **this does not scale.**

What is Wrong with This?

- Data (`playerHp`, `playerDamage`) and behavior (`attack`) are **separate**
- Every method needs the data **passed in as a parameter**
- Adding a new property (e.g., `mana`) means changing **every** function signature
- No way to say "this name belongs to this HP belongs to this damage"
- Easy to mix up: `attack(enemyHp, playerArmor, playerDamage)` -- is that right?

What is Wrong with This?

- Data (`playerHp`, `playerDamage`) and behavior (`attack`) are **separate**
- Every method needs the data **passed in as a parameter**
- Adding a new property (e.g., `mana`) means changing **every** function signature
- No way to say "this name belongs to this HP belongs to this damage"
- Easy to mix up: `attack(enemyHp, playerArmor, playerDamage)` -- is that right?

Predict the output

If someone writes `attack(enemyHp, playerArmor, playerDamage)` instead of `attack(enemyHp, playerDamage, playerArmor)` -- what happens? Does the compiler catch this?

The core issue

Data that **belongs together** (a player's name, HP, damage) is scattered across **unrelated variables**. We need a way to group them.

The OOP Idea

Object-Oriented Programming bundles data and behavior together:

Procedural (Modern Code)

- Data: standalone variables
- Behavior: static methods
- Connection: parameters
- "I have HP, damage, and a function that takes HP and damage"

Object-Oriented (this semester)

- Data: fields inside an object
- Behavior: methods on the object
- Connection: automatic (built-in)
- "I have a Player that *knows* its HP and can attack"

Side by Side: Procedural vs OOP

Procedural

```
String name = "Hero";
int hp = 100;
int damage = 25;

static int attack(
    int targetHp, int dmg) {
  return Math.max(0,
    targetHp - dmg);
}

// must pass everything
enemyHp = attack(
    enemyHp, damage);
```

Object-Oriented

```
class Player {
  String name;
  int hp;
  int damage;
  Player(String name, int hp, int damage)
    {
    this.name = name;
    this.hp = hp;
    this.damage = damage;
  }
  void attack(Player target) {
    target.hp -= this.damage;
  }
}
Player hero = new Player("Hero", 100, 25);
hero.attack(goblin);
```

Key Differences at a Glance

Concept	Procedural (Sem. 1)	OOP (Sem. 2)
Data	standalone variables	fields in a class
Behavior	`static` methods	instance methods
Data flow	pass as parameters	object knows via `this`
Creating	`int hp = 100;`	`new Player("Hero", 100, 25)`
Calling	`attack(enemyHp, dmg)`	`hero.attack(goblin)`
Grouping	one class, many functions	one class per concept

The **same Java language** -- just a different way of structuring your code.

Quick poll: Which row in this table feels like the biggest change for you?

Classes: Blueprints for Objects

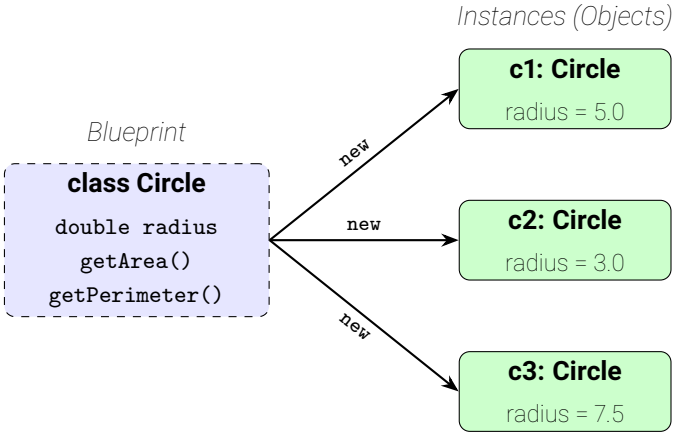
A **class** is a template that defines:

- **Fields** (instance variables) -- the data each object holds
- **Constructors** -- how to initialize new objects
- **Methods** -- the behavior each object can perform

Analogy

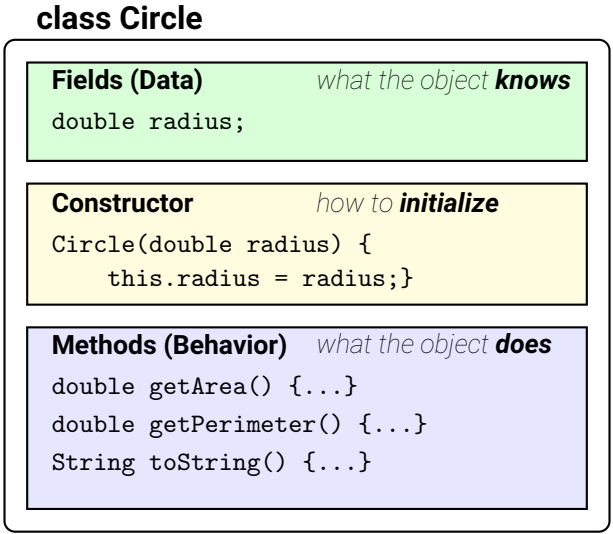
A class is like a **cookie cutter** -- it defines the shape.
An object is like a **cookie** -- it is a concrete instance made from that shape.
You can make many cookies from one cutter, each with different decorations.

One Class, Many Objects



One class definition → many independent objects, each with their own field values.

Anatomy of a Class



Defining a Class

The `class` keyword creates a new type:

```
1   class Circle {
2       // everything about a Circle goes in here
3   }
```

- `class Circle { }` defines a new type called `Circle`
- The curly braces contain everything that belongs to a `Circle`: its data and its behavior
- Convention: class names start with an **uppercase letter** (`Circle`, `Rectangle`, `Player`)
- **One class per file**, filename matches class name: `Circle.java`

Fields: Data Inside a Class

Fields are variables that belong to each object (not to a method):

```
1   class Circle {
2       double radius;  // field: each Circle has its own radius
3   }
```

- In Modern Code: `double radius = 5.0;` was a **local variable** in a method
- Now: `double radius;` is a **field** that lives inside the object
- Each object gets its own copy of the fields

Field Default Values

Fields get **default values** if you do not set them in the constructor:

Type	Default Value	Example
`int`, `long`	`0`	HP starts at 0, not 100
`double`, `float`	`0.0`	Radius is 0.0, not useful
`boolean`	`false`	Player is “not alive”
`String`, objects	`null`	Name is missing entirely

These are the values Java assigns automatically if you do **not** initialize a field yourself. Unlike local variables (which cause a compiler error), fields silently get these defaults.

Initializing Fields

You can give fields a value directly where you declare them:

```
1   class Circle {
2       double radius = 1.0;  // every new Circle starts with radius 1.0
3   }
```

- This overrides the default value (`0.0`) with something meaningful
- Every `Circle` object created will start with `radius = 1.0`
- But what if we want different circles with different radii?
- → That is what **constructors** are for (next topic)

Why Constructors Matter

Without a constructor, objects start with **useless defaults**:

```
class Enemy {
    String name;    // default: null
    int hp;         // default: 0
}
Enemy e = new Enemy();
IO.println(e.name);   // null
IO.println(e.hp);     // 0
```

Easy Initialization

A constructor's job is to replace these useless defaults with **meaningful values**. Every class you write should have a constructor that sets up valid initial state.

Constructors: Creating Objects

A **constructor** initializes an object when it is created with `new`:

```
1  class Circle {
2      double radius;
3      Circle(double radius) {
4          this.radius = radius;
5      }
6  }
```

- Constructor name = class name (no return type!)
- Called automatically when you write `new Circle(5.0)`
- Sets up the object's initial state
- Without a constructor, Java provides a **default constructor** that takes no arguments and sets all fields to their defaults

Multiple Constructors

Like method overloading from Modern Code -- a class can have **multiple constructors**:

```
class Player {
    String name;
    int hp;

    Player(String name, int hp) {
        this.name = name;
        this.hp = hp;
    }

    Player(String name) {
        this(name, 100);  // delegates to the other constructor
    }
```

```
Player custom = new Player("Hero", 200);
Player quick  = new Player("Villager");  // hp=100
```

Note: Both constructors create the same *kind* of object -- they just offer different ways to initialize it.

The `this` Keyword

`this` refers to the **current object** -- the object whose method is being called:

```
1   class Circle {
2       double radius;
3
4       Circle(double radius) {
5           this.radius = radius;
6       }
7   }
```

- `this.radius` -- the object's field
- `radius` (without `this`) -- the constructor parameter
- `this` resolves the **name conflict** between field and parameter
- In Modern Code you passed data as parameters; now the object *knows* its data via `this`

`this.radius` vs `radius` -- Not the same!

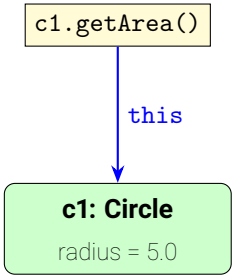
```
1    class Circle {
2        double radius;
3
4        Circle(double radius) {
5            this.radius = radius;
6            radius = 0; // changes
         parameter
7            // this.radius is unchanged!
8        }
9    }
10
11   Circle c = new Circle(5.0);
12   // c.radius is still 5.0
```

What happens:

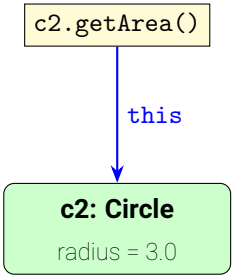
1. Parameter `radius` = `5.0`
2. `this.radius = radius;` copies `5.0` into the field
3. `radius = 0;` sets the *parameter* to 0
4. The *field* `this.radius` is untouched -- still `5.0`

They are **two separate variables** that happen to have the same name. `this.` is how you tell Java which one you mean.

How `this` Works



$\pi \times 5.0^2 = 78.54$



$\pi \times 3.0^2 = 28.27$

`this` always refers to the object **left of the dot**.

`c1.getArea()` → inside that call, `this` = `c1`

`c2.getArea()` → inside that call, `this` = `c2`

Quick Check: Predict the Output

```
1   Circle c1 = new Circle(5.0);
2   Circle c2 = new Circle(3.0);
3   IO.println(c2.getArea());
```

What does this print?

Quick Check: Predict the Output

```
1   Circle c1 = new Circle(5.0);
2   Circle c2 = new Circle(3.0);
3   IO.println(c2.getArea());
```

What does this print?

Answer

`28.274...` ($\pi \times 3.0^2$). When `c2.getArea()` runs, `this` = `c2`, so `this.radius` is `3.0`. The value of `c1.radius` (5.0) is irrelevant -- each object has its own fields.

`this` is Optional (When Unambiguous)

If there is **no name conflict**, you can omit `this`:

```
1   class Circle {
2       double radius;
3       // "this" required: parameter name = field name
4       Circle(double radius) {
5           this.radius = radius;
6       }
7       // "this" optional: no name conflict
8       double getArea() {
9           return Math.PI * radius * radius;
10          // same as: Math.PI * this.radius * this.radius
11      }
12  }
```

Recommendation

Always use `this` it makes the code clearer.

Instance Methods: Behavior

Instance methods operate on the object's own data -- no `static`, no extra parameters:

Modern Code (static)

```
1  static double calcArea(
2          double radius) {
3      return Math.PI
4          * radius * radius;
5  }
6  // must pass radius in
7  double a = calcArea(5.0);
```

OOP (instance method)

```
1  double getArea() {
2      return Math.PI
3          * this.radius
4          * this.radius;
5  }
6  // object knows its radius
7  double a = c.getArea();
```

-- No `static` keyword -- the method belongs to the **object**, not the class

-- No `radius` parameter -- the object accesses its own field via `this`

A Complete Circle Class

Without looking back: what are the three main parts of a class? … Now let's see the complete class and check.

A Complete Circle Class

Without looking back: what are the three main parts of a class? … Now let's see the complete class and check.

```
1   class Circle {
2       double radius;
3
4       Circle(double radius) {
5           this.radius = radius;
6       }
7
8       double getArea() {
9           return Math.PI * this.radius * this.radius;
10      }
11      double getPerimeter() {
12          return 2 * Math.PI * this.radius;
13      }
14  }
```

Creating Objects with `new`

The `new` keyword creates an object (an **instance**) from a class:

```
1   Circle c1 = new Circle(5.0);
2   Circle c2 = new Circle(3.0);
```

What happens when you write `new Circle(5.0)`:

1. Java allocates memory for a new Circle object
2. The constructor `Circle(5.0)` runs and sets `this.radius = 5.0`
3. A **reference** to the new object is returned and stored in `c1`

- `c1` and `c2` are two **different objects** -- each has its own `radius`
- You can create as many objects from one class as you need

Using Objects: Methods

Call methods on an object with the **dot operator**:

```
1   Circle c = new Circle(5.0);
2
3   double area = c.getArea();           // 78.539...
4   double perimeter = c.getPerimeter(); // 31.415...
5   String text = c.toString();          // "Circle(radius=5.0)"
```

- `c.getArea()` -- calls `getArea()` on the object `c`
- Inside `getArea()`, `this` refers to `c`

Using Objects: Fields

You can also access an object's **fields** with the dot operator:

```
1   Circle c = new Circle(5.0);
2
3   // Access field directly (for now)
4   IO.println("Radius: " + c.radius);
```

- `c.radius` reads the `radius` field of the object `c`
- We access fields directly for now -- later we will learn why this can be problematic

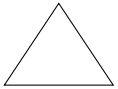
Other Shapes Work the Same Way

The same pattern applies to any shape -- for example, a **`Rectangle`**:

- **Fields:** `double width;` and `double height;`
- **Constructor:** `Rectangle(double width, double height)`
- **Methods:** `getArea()`, `getPerimeter()`, `toString()`

The structure is identical to **`Circle`** -- just different fields and calculations.

Thought Experiment: Defining Shapes



Triangle

Star

What data do you need to create each shape?

Thought Experiment: Triangle

One way to define it --- same pattern as `Circle` and `Rectangle`:

- **Fields:** e.g. `double base;`, `double height;` (or three side lengths `a`, `b`, `c`)
- **Constructor:** `Triangle(double base, double height)` (or `Triangle(double a, double b, double c)`)
- **Methods:** `getArea()`, `getPerimeter()`, `toString()`

Other valid choices (e.g. triangle from three vertices); the important part: **state in fields**, **behavior in methods**.

Thought Experiment: Star

One way to define it --- same pattern:

- **Fields:** e.g. `double outerRadius;`, `int points;` (number of tips), `innerRadius` or ratio
- **Constructor:** `Star(double outerRadius, int points)` (and maybe inner radius/ratio)
- **Methods:** `getArea()`, `getPerimeter()`, `toString()`

Same idea: **state in fields**, **behavior in methods**.

Working with Multiple Objects

You can create many objects from the same class; each has its own state:

```
1   Circle small = new Circle(1.0);
2   Circle big = new Circle(10.0);
3
4   IO.println(small.getArea());  // 3.14159...
5   IO.println(big.getArea());    // 314.159...
```

- **small** and **big** are two separate objects in memory
- Each has its own **radius** field

Objects Are Independent

Changing one object does not affect the other:

```
small.radius = 2.0;
IO.println(small.getArea());  // 12.566...
IO.println(big.getArea());    // 314.159... (unchanged)
```

-- Modifying **small** does not change **big**

Objects are References

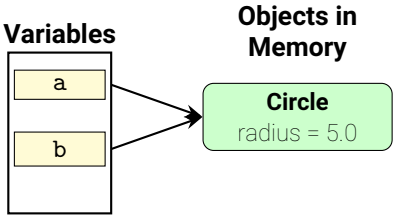
Think of it like a house address: `Circle b = a;` doesn't build a new house -- it copies the **address**. Both variables now point to the same house. Any changes through one address are visible through the other (same as with arrays).

```
1    Circle a = new Circle(5.0);
2    Circle b = a;          // b points to the SAME object
3    b.radius = 10.0;
4    IO.println(a.radius);  // 10.0 -- a and b are the same object!
5
6    Circle c = new Circle(5.0);
7    Circle d = new Circle(5.0);  // d is a DIFFERENT object
8    d.radius = 10.0;
9    IO.println(c.radius);  // 5.0 -- c and d are different objects
```

- `Circle b = a;` copies the **reference**, not the object
- `new` always creates a **new, separate object**

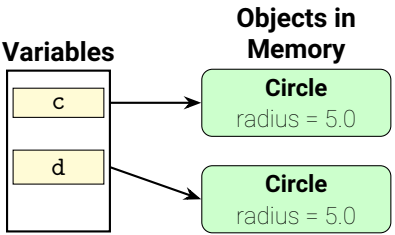
References in Memory

Shared reference: `Circle b = a;`



Same object -- changing `b.radius` also changes `a.radius`

Separate objects: `new` twice



Different objects -- independent state

(Variables are on the Stack / Objects are on the Heap)

Quick Check: References

```
1  Circle x = new Circle(5.0);
2  Circle y = new Circle(5.0);
3  y.radius = 10.0;
4  IO.println(x.radius);
```

What does this print? `5.0` or `10.0`?

Quick Check: References

```
1  Circle x = new Circle(5.0);
2  Circle y = new Circle(5.0);
3  y.radius = 10.0;
4  IO.println(x.radius);
```

What does this print? `5.0` or `10.0`?

Answer

`5.0` -- because `new` creates a **separate object**. Even though both start with the same radius, `x` and `y` are independent objects. Compare this to `Circle b = a;` on the previous slides, where both variables share the **same** object.

The `toString()` Method

`toString()` provides a human-readable text representation of an object:

```
1  class Circle {
2      double radius;
3      // constructor omitted for brevity
4
5      String toString() {
6          return "Circle(radius=" + this.radius + ")";
7      }
8  }
```

-- Returns a `String` describing the object's state

Using `toString()`

```
1   Circle c = new Circle(5.0);
2   IO.println(c.toString());  // "Circle(radius=5.0)"
3   IO.println(c);             // also calls toString() automatically
```

- Without **`toString()`**: prints something like **`Circle@1a2b3c4`**
- Good **`toString()`** methods are invaluable for debugging

Methods That Take Parameters

Instance methods can take parameters for data that is **not** part of the object:

```
1   class Circle {
2       double radius;
3       // constructor omitted for brevity
4
5       boolean isBiggerThan(Circle other) {
6           return this.getArea() > other.getArea();
7       }
8
9       Circle scale(double factor) {
10          return new Circle(this.radius * factor);
11      }
12  }
```

- `isBiggerThan(Circle other)` -- compares with another circle
- `scale(double factor)` -- returns a new circle, does not change `this`

Using Methods With Parameters

```
1   Circle c1 = new Circle(5.0);
2   Circle c2 = new Circle(3.0);
3   IO.println(c1.isBiggerThan(c2));  // true
4
5   Circle c3 = c1.scale(2.0);          // new Circle with radius 10.0
```

Notice: `scale` creates a **new** Circle -- it does not change `c1`. After this line, `c1.radius` is still `5.0`.

Objects as Method Parameters

You can pass objects to methods -- just like **`int`** or **`double`**:

```
1   class Player {
2       String name;
3       int hp;
4       int damage;
5       // constructor omitted
6
7       void attack(Player target) {
8           target.hp = Math.max(0, target.hp - this.damage);
9           IO.println(this.name + " attacks " + target.name
10              + " for " + this.damage + " damage!");
11      }
12  }
```

- **`attack(Player target)`** receives another **`Player`** and can change **`target.hp`**

Passing Objects: The Method Can Modify Them

```
1  Player hero   = new Player("Hero", 100, 25);
2  Player goblin = new Player("Goblin", 50, 10);
3  hero.attack(goblin);   // "Hero attacks Goblin for 25 damage!"
4  IO.println(goblin.hp); // 25
```

Important

`hero.attack(goblin)` changed `goblin`'s HP. When you pass an object to a method, the method can modify it -- because it receives the **reference**, not a copy.

Step by Step: What Happens During `hero.attack(goblin)`?

Let's trace through the method call line by line:

1. Java sees `hero.attack(...)` → `this` = `hero`
2. The argument `goblin` is assigned to the parameter `target`
3. `this.damage` = hero's damage = `25`
4. `target.hp = Math.max(0, 50 - 25)` = `25`
5. `goblin.hp` is now `25` -- the original object was modified

```
1   void attack(Player target) {      // target = goblin
2       target.hp = Math.max(0,
3           target.hp - this.damage); // this = hero
4   }
```

The `null` Reference

A variable can hold **`null`** -- meaning "no object":

```
1   Circle c = null;        // c points to nothing
2   IO.println(c.getArea()); // CRASH: NullPointerException!
```

- **`null`** means "no object" -- the variable refers to nothing
- Calling a method on **`null`** causes a crash

NullPointerException

NullPointerException (NPE) is one of the most common Java runtime errors. It means: "You tried to use an object that does not exist."

```
1  // Common cause: forgetting to create the object
2  Circle c;                    // c is null (not initialized)
3  c.getArea();                 // CRASH!
4
5  // Fix: always create the object first
6  Circle c = new Circle(5.0);  // c points to a real object
7  c.getArea();                 // works fine
```

Arrays of Objects

You can create arrays of objects -- just like arrays of **`int`**:

```
1   // Array of 3 Circle objects
2   Circle[] circles = new Circle[3];
3   circles[0] = new Circle(1.0);
4   circles[1] = new Circle(2.0);
5   circles[2] = new Circle(3.0);
```

`new Circle[3]` creates an array of 3 slots; each slot defaults to **`null`**. You must create each object with **`new Circle(...)`** before using it.

Your Turn: Design a Weapon Class

Interactive break

Think about this for 2 minutes

You are designing a weapon system for an RPG. What **fields** and **methods** would a `Weapon` class need?

Consider:

- What data defines a weapon? (name, damage, type, durability, ...)
- What can a weapon *do*? (deal damage, break, describe itself, ...)
- Should a weapon know about the player using it, or just about itself?

Bonus: How would `Player` and `Weapon` interact?
Could a Player *have* a Weapon? (Hint: a field of type `Weapon`)

Discuss with your neighbor -- we will compare ideas in a moment.

One Possible Weapon Design

(One) Answer

```
1   class Weapon {
2       String name;
3       int damage;
4       int durability;
5
6       Weapon(String name, int damage, int durability) {...}
7
8       int use() {
9           if (this.durability <= 0) {
10              return 0;
11          }
12          this.durability--;
13          return this.damage;
14      }
15
16      boolean isBroken() { return this.durability <= 0; }
17
18      String toString() {...}
19  }
```

Combining Objects: Player with Weapon

A class can have **fields that are objects** -- objects inside objects:

```
1   class Player {
2       String name;
3       int hp;
4       Weapon weapon;  // a Player HAS a Weapon
5
6       Player(String name, int hp, Weapon weapon) {\dots}
7
8       void attack(Player target) {
9           int dmg = this.weapon.use();
10          target.hp = Math.max(0, target.hp - dmg);
11      }
12  }
```

- `weapon` is a **reference** to a `Weapon` object
- `this.weapon.use()` delegates to the weapon's method

Using Player with Weapon

```
1   Weapon sword = new Weapon("Iron Sword", 20, 10);
2   Player hero = new Player("Hero", 100, sword);
3   hero.attack(goblin);  // Hero uses sword to attack
```

Preview: This pattern -- objects containing other objects -- is called *composition*. We will practice it more in later lectures.

Putting It All Together

A complete mini-program -- creating objects, calling methods, seeing results:

```
1   public class Game {
2       public static void main(String[] args) {
3           Weapon sword  = new Weapon("Iron Sword", 20, 3);
4           Player hero   = new Player("Hero", 100, sword);
5           Player goblin = new Player("Goblin", 50,
6                                      new Weapon("Club", 10, 2));
7
8           IO.println(hero);       // Hero (100 HP, Iron Sword)
9           IO.println(goblin);     // Goblin (50 HP, Club)
10          hero.attack(goblin);    // Hero attacks Goblin for 20 dmg
11          IO.println(goblin.hp); // 30
12          goblin.attack(hero);    // Goblin attacks Hero for 10 dmg
13          IO.println(hero.hp);    // 90
14      }
15  }
```

`main` creates the objects, but the **logic lives inside the classes**. Each object knows its own data and behavior.

Spot the Bug!

Interactive break

Two code snippets, two bugs. Can you find them? We'll reveal in a moment.

Spot the Bug! (1)

Snippet 1

```
class Circle {
    double radius;
    Circle(double radius) {
        radius = radius;
    }
}
Circle c = new Circle(5.0);
IO.println(c.radius); // ???
```

Spot the Bug! (2)

Snippet 2

```
1   class Circle {
2       double radius;
3       Circle(double radius) {
4           this.radius = radius;
5       }
6       static double getArea() {
7           return Math.PI
8               * this.radius
9               * this.radius;
10      }
11  }
```

Common Mistakes

(One) Answer)

Bug 1: Forgetting `this`

```
1  Circle(double radius) {
2      this.radius = radius;
3      // fix: was radius =
       radius
4  }
```

Bug 2: Using `static` on methods

```
1  double getArea() {
2    // fix: remove static
3      return Math.PI * this.
       radius * this.radius;
4  }
```

Common Mistakes (continued)

Return type on constructor

```
1  // WRONG: this is a method,
2  // not a constructor!
3  void Circle(double radius)
       {
4      this.radius = radius;
5  }
```

Fix: Remove `void`.
Constructors have no return type.

Comparing objects with `==` (same as with Strings)

```
1  Circle a = new Circle(5.0);
2  Circle b = new Circle(5.0);
3  IO.println(a == b); //
       false!
4  // == compares references,
5  // not field values
```

`==` checks if two variables point to the **same object**, not if they have equal fields.

*We will learn the proper way to compare objects (**`equals`**) in a later lecture.*

When to Use Classes

Create a class when you have **data and behavior that belong together**:

Good candidates for classes

- Circle (radius + area/perimeter)
- Player (name, hp + attack/heal)
- Weapon (name, damage + use)
- BankAccount (balance + deposit)
- Enemy (name, hp + drop loot)

Still fine as static methods

- `Math.sqrt()` -- pure utility
- `Integer.parseInt()` -- conversion
- Helper functions with no state
- `main()` -- program entry point

Rule of thumb: If you find yourself passing the same data to multiple related functions, those functions and that data probably belong in a class.

Class Template: Your Starting Point

```
class MyClass {
    // 1. Fields (data)
    type fieldName;

    // 2. Constructor (initialization)
    MyClass(type fieldName) {
        this.fieldName = fieldName;
    }

    // 3. Methods (behavior)
    returnType methodName() {
        // use this.fieldName
    }
}
```

Every class follows this structure: fields at the top, then constructor(s), then methods. When you start an exercise, copy this template and fill in the blanks.

Quick Recap: What We Have Learned So Far

Before we move on to testing, let us make sure the key ideas are clear:

- A **class** bundles data (fields) and behavior (methods) together
- A **constructor** initializes objects -- called via `new`
- `this` refers to the **current object** (the one left of the dot)
- Objects are **references** -- assigning copies the reference, not the object
- Objects can contain other objects (**composition**)

Any questions before we continue?

From Manual Testing to JUnit

Modern Code approach

- Print output to console
- Visually check if it looks right
- “Looks correct to me!”
- Run the program again after changes
- Easy to miss broken things

JUnit approach

- Write test methods
- **Assert** expected values
- Computer checks automatically
- Run all tests with one click
- Instantly see what broke

In Modern Code you tested by printing and checking manually.
Now we **formalize** this into repeatable, automated tests.

JUnit 5 Basics

```
1   class CircleTest {
2
3       @Test
4       void testGetArea() {
5           Circle c = new Circle(5.0);
6           assertEquals(78.5398, c.getArea(), 0.001);
7       }
8   }
```

- `@Test` -- marks a method as a test (JUnit runs it automatically)
- `assertEquals(expected, actual, delta)` -- checks if two doubles are equal within a tolerance (`delta`)
- The `delta` handles floating-point rounding (e.g., `0.001`)

Annotations

In Java, an **annotation** is a special label that you put on classes, methods, or fields. It gives extra information to the compiler or to tools and frameworks.

- Written with an `@` symbol, e.g. `@Test`, `@Override`
- They don't change your code's logic by themselves -- they are *metadata*
- Frameworks (like JUnit) read them and behave accordingly

For now: think of `@Something` as a tag that says "treat this in a special way."

The `@Test` Annotation

```
1   class CircleTest {
2       @Test
3       void testGetArea() {
4           // This method will be run as a test
5       }
6
7       void helperMethod() {
8           // NO @Test -> this is NOT a test, just a helper
9       }
10  }
```

- **`@Test`** tells JUnit: "run this method as a test"
- Test methods must be **`void`** and take no parameters
- Methods without **`@Test`** are ignored by JUnit

Why `delta` Matters for Doubles

Quick question: What do you think `0.1 + 0.2` equals in Java? Is it exactly `0.3`?

Why `delta` Matters for Doubles

Quick question: What do you think `0.1 + 0.2` equals in Java? Is it exactly `0.3`?

Floating-point math is not exact -- small rounding errors are normal:

```
1   double result = 0.1 + 0.2;
2   IO.println(result);          // 0.30000000000000004
3   IO.println(result == 0.3);   // false!
```

Using `delta` in Tests

That is why `assertEquals` for doubles uses a **delta** (tolerance):

```
@Test
void testGetArea() {
    Circle c = new Circle(1.0);
    // Without delta: might FAIL due to rounding
    // assertEquals(3.14159265, c.getArea());
    // With delta: passes if difference < 0.0001
    assertEquals(Math.PI, c.getArea(), 0.0001);
}
```

Rule: Always use the 3-argument `assertEquals` when comparing `double` values.

Writing More Tests

One test class usually has **several** `@Test` methods -- one per behavior you want to check. Use clear names: `testGetArea`, `testGetPerimeter`, etc. **Note:** the execution order of the test methods is not guaranteed.

```
1   class CircleTest {
2       @Test
3       void testGetArea() {
4           Circle c = new Circle(1.0);
5           assertEquals(Math.PI, c.getArea(), 0.0001);
6       }
7       @Test
8       void testGetPerimeter() {
9           Circle c = new Circle(1.0);
10          assertEquals(2 * Math.PI, c.getPerimeter(), 0.0001);
11      }
12      @Test
13      void testToString() {...}
14  }
```

Setup and Teardown:

Instead of creating the same object in every test, use **lifecycle** methods:

```
1   class CircleTest {
2       Circle c;
3
4       @BeforeEach
5       void setUp() {
6           c = new Circle(5.0);   // fresh circle before each test
7       }
8
9       @Test
10      void testGetArea() {...}
11      @Test
12      void testGetPerimeter() {...}
13  }
```

- `@BeforeEach` runs **before each** `@Test`; `@AfterEach` runs after (e.g. close a file).
- Order: BeforeEach → Test → AfterEach (AfterEach runs even if the test fails).

`@BeforeAll` and `@AfterAll`

For setup that is **expensive** (Database Connection, ...) or shared by all tests, run it once per test class:

```
1   class DataFileTest {
2       static Path testData;
3
4       @BeforeAll
5       static void loadOnce() {
6           testData = Paths.get("testdata.csv");
7       }
8       @AfterAll
9       static void cleanup() {
10          // e.g. delete temp files
11      }
12      ...
13  }
```

- `@BeforeAll` runs **once** before any test in the class; `@AfterAll` runs once after all tests.
- Both methods must be `static` (no per-test instance yet).

Common Assertions

JUnit provides several assertion methods:

Assertion	**Checks that...**
`assertEquals(exp, act)`	values are equal
`assertEquals(exp, act, delta)`	doubles are equal within tolerance
`assertTrue(condition)`	condition is true
`assertFalse(condition)`	condition is false
`assertNotNull(object)`	object is not null
`assertNull(object)`	object is null
`assertThrows(ExType.class, () -> ...)`	code throws expected exception

Testing Exceptions with `assertThrows`

When a method should throw (e.g. invalid input), assert that it does:

```
1   @Test
2   void testNegativeRadiusThrows() {
3       assertThrows(IllegalArgumentException.class,
4           () -> new Circle(-1.0));
5   }
6
7   @Test
8   void testBrokenWeaponReturnsZero() {
9       Weapon w = new Weapon("Sword", 10, 0);
10      assertEquals(0, w.use());  // durability 0 -> 0 damage
11  }
```

- `assertThrows(ExceptionType.class, () -> code)` -- run `code`; test passes only if that exception is thrown.
- Use for validation (e.g. negative radius) or for behavior that must not throw (e.g. `use()` returns 0 when broken).

Test Structure: Given -- When -- Then (Recap)

From Modern Code (Semester 1): every good test follows the same pattern:

1. **Given** -- set up the objects and data you need
2. **When** -- call the method you want to test
3. **Then** -- check that the result is correct

```
@Test
void testGetArea() {
    // Given
    Circle c = new Circle(5.0);
    // When
    double area = c.getArea();
    // Then
    assertEquals(78.5398, area, 0.001);
}
```

What Happens When a Test Fails?

JUnit shows you **exactly what went wrong**:

```
@Test
void testGetArea() {
    Circle c = new Circle(5.0);
    assertEquals(100.0, c.getArea(), 0.001);  // WRONG expected
}
```

JUnit output:

```
org.opentest4j.AssertionFailedError:
expected: <100.0> but was: <78.53981633974483>
    at CircleTest.testGetArea(CircleTest.java:7)
```

Testing Edge Cases

Good tests check not just the **happy path**, but also **edge cases** -- inputs or situations at the boundaries (zero, empty, very large, or invalid).

- **Examples:** radius = 0, negative values, damage greater than current HP, empty strings
- The goal: make sure your code behaves correctly (or fails clearly) in these situations
- Use descriptive test names: `testZeroRadius`, `testDamageExceedsHP`, etc.

JUnit Extra: `@DisplayName`

Use `@DisplayName("...")` to show a **human-readable name** in test reports and in the IDE instead of the method name.

```
1    @Test
2    @DisplayName("Area is pi for radius 1")
3    void testGetArea() {
4        Circle c = new Circle(1.0);
5        assertEquals(Math.PI, c.getArea(), 0.0001);
6    }
```

When the test runs, the report shows “Area is pi for radius 1” instead of `testGetArea` -- useful when method names are technical or abbreviated.

JUnit Extra: `assertAll`

Normally, the first failed assertion stops the test. With **`assertAll`**, JUnit runs **all** supplied assertions and then reports every failure.

```
1   @Test
2   void testCircle() {
3       Circle c = new Circle(5.0);
4       assertAll(
5           () -> assertEquals(78.54, c.getArea(), 0.01),
6           () -> assertEquals(31.42, c.getPerimeter(), 0.01),
7           () -> assertFalse(c.getRadius() < 0)
8       );
9   }
```

Useful when one test checks several properties: you see all mismatches at once instead of fixing one and re-running.

Note: The `() -> ...` syntax is a **lambda expression** (a small anonymous block of code). We will learn about lambdas properly later in the course;.

JUnit Extra: `@Disabled`

Temporarily **skip** a test without deleting it: add `@Disabled` (or `@Disabled("reason")`).

```
1    @Test
2    @Disabled("Waiting for new API")
3    void testNewFeature() {
4        // ...
5    }
6
7    @Test
8    @Disabled
9    void testBrokenTemporarily() {
10       // ...
11   }
```

JUnit will not run the method; the report shows it as disabled. Remove `@Disabled` when the test is ready again.

Interactive break

Your Turn: What Would You Test?

Think about this for 2 minutes

Given the **`Weapon`** class from earlier:

- `Weapon(String name, int damage, int durability)`
- `int use()` -- returns damage and decreases durability
- `boolean isBroken()` -- true if durability ≤ 0

What test methods would you write?

- What is the "happy path" test?
- What edge cases should you check?
- What happens if you use a broken weapon?
- What about a weapon with durability = 1?

Discuss with your neighbor -- what did you come up with?

Possible Weapon Tests

(One) Answer

One possible set of tests:

- `testUseReturnsDamage` -- Given a new weapon, When `use()`, Then return correct damage
- `testUseDecreasesDurability` -- After `use()`, durability is reduced by 1
- `testBrokenWeaponReturnsZero` -- Given a broken weapon (durability 0), When `use()`, Then returns 0
- `testNewWeaponIsNotBroken` -- Given a new weapon, When we check `isBroken()`, Then false
- `testUsedWeaponIsBrokenWhenDurabilityZero` -- Given a weapon with durability 1, When `use()`, Then `isBroken()` is true

Advanced JUnit Features (You Probably Won't Need)

For this course, the basics are enough. If you read other code or do larger projects, you may see:

- `@ParameterizedTest` -- run the same test with many inputs (e.g. `@CsvSource`); avoids copy-pasting similar tests.
- `@Nested` -- group related tests in inner classes with shared setup.
- `assertTimeout` / `assertTimeoutPreemptively` -- fail if code runs longer than a given time (useful for algorithm performance).
- **Assumptions** (`assumeTrue`, `assumeFalse`) -- run a test only if a condition holds (e.g. "only on Windows").

No need to learn these now; the previous slides cover what you need 99% of the time.

When to Write Tests

- Write tests for **every public method** that computes or changes something
- Test the **normal case** first, then think about edge cases
- Tests are code too -- keep them **simple and readable**
- Run tests **after every change** to catch regressions

Benefits of testing

- You find bugs **before** the deadline, not during grading
- You can refactor code **with confidence** -- tests catch mistakes
- Tests serve as **documentation** -- they show how the class is meant to be used
- In this course: assignments are tested with JUnit -- write your own tests first!

Quick Retrieval: Test Yourself

Before looking at the summary -- can you answer these from memory?

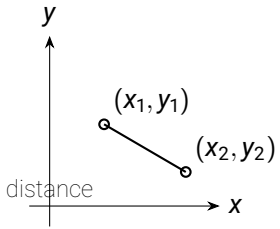
1. Name 3 differences between procedural and object-oriented code.
2. What does **`this`** refer to inside an instance method?
3. What happens if you write **`Circle c;`** without **`new`**?
4. What does **`@Test`** do in JUnit?

Take some time to think, then we go through the answers.

Summary

- A **class** bundles data (fields) and behavior (methods) together
- A **constructor** initializes objects; called via `new Circle(5.0)`
- `this` refers to the current object -- replaces passing data as parameters
- Instance methods have **no** `static` -- they operate on the object's own fields
- Objects are **references** (like arrays) -- assigning copies the reference, not the object
- **JUnit** tests formalize testing: `@Test` + `assertEquals`

Appendix: Point



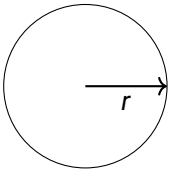
Fields: x, y (position in the plane).

distanceTo(other): Euclidean distance between this point and `other`:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Implement `distanceTo(Point other)` using this formula.

Appendix: Circle



radius ***r***

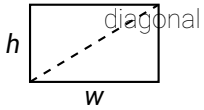
Field: `radius` *r*.

getArea(): $A = \pi r^2$

getPerimeter(): $P = 2\pi r$ (circumference)

scale(factor): new radius = $r \cdot$ factor. Return a *new* circle; do not modify `this`.

Appendix: Rectangle



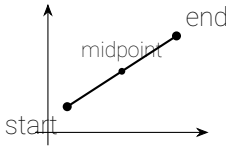
Fields: `width` w, `height` h.

getArea(): $A = w \cdot h$

getPerimeter(): $P = 2(w + h)$

getDiagonal(): $d = \sqrt{w^2 + h^2}$ (Pythagorean theorem)

Appendix: Line



Fields: `start`, `end` (both `Point`).

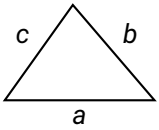
getLength(): length = distance from `start` to `end` (reuse `Point.distanceTo`).

getMidpoint(): midpoint coordinates

$$\left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2}\right)$$

Return a new `Point`.

Appendix: Triangle



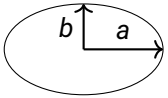
Fields: `sideA` a, `sideB` b, `sideC` c.

getPerimeter(): $P = a + b + c$

getArea(): Heron's formula. Semi-perimeter $s = P/2$, then

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

Appendix: Ellipse



semi-axes a, b

Fields: `semiMajorAxis` a, `semiMinorAxis` b.

getArea(): $A = \pi ab$

getPerimeter(): No exact closed form. Ramanujan approximation:

$$P \approx \pi\left(3(a+b) - \sqrt{(3a+b)(a+3b)}\right)$$

Appendix: Star

n points

Fields: `numPoints` n, `outerRadius` R, `innerRadius` r.

getArea():

$$A = n \cdot R \cdot r \cdot \sin(\pi/n)$$

getPerimeter(): Sum of all edge lengths. Each edge can be computed from R, r, and the angle step π/n (e.g. law of cosines for side length).